ScriptLens — Plain-Language Review of the Two Engines

What this is	A simple, jargon-free explanation of what your two analysis engines do, how good they are right now, where they fall short, and what to do next.
Date	June 13, 2026
Who it's for	You — to decide what to work on, in plain English.

---

1. The big picture (in one paragraph)

ScriptLens reads a screenplay and tries to do two helpful things for a writer: it **maps how scenes connect** (so you know what breaks if you cut a scene), and it **catches story mistakes** (like a character being dead in one scene and alive later). Both of these work today and pass their own tests. The catch is that they mostly work by **looking for specific word patterns**. That means they are accurate on the examples they were built for, but they **miss things written in unexpected ways**, and once in a while they **flag something that isn't really a mistake**. The most valuable thing you can do next is **build a small collection of real scripts with the "right answers" marked**, because that's the only way to safely improve the trickier parts.

---

2. What the two engines actually do

Engine 1 — "Scene Connections" (scene dependency)

Think of this as drawing a map of your story. It reads each scene and notes: - **Who's in it** (characters), - **What important objects show up** (props, like a briefcase or a key), - **Where it takes place** (location).

Then it draws arrows: if a character, prop, or place from an early scene shows up again later, it links those scenes. With this map it can tell you: - "If you delete this scene, these later scenes lose their setup." - "These scenes are 'orphans' — nothing later depends on them, so they may be easy to cut." - "These are your most important scenes — don't cut them lightly."

Engine 2 — "Story Mistake Finder" (plot contradiction)

This one reads the script looking for facts it can pin down, such as: - "This character is dead." - "Today is Monday." - "This person is a surgeon." - "Elena is holding the ledger."

Then it checks whether the script later **contradicts** any of those facts — for example, a character who was killed earlier suddenly speaks again, or the days of the week don't add up. It sorts findings by how confident it is.

After both engines run, the system also gives the script an overall **health score out of 100**.

---

3. Current status — how good is it right now?

Scene connections	**Working**	Builds the map and answers "what breaks if I cut this?" reliably on test scripts.
Story mistake finder	**Working**	Catches the planted mistakes in the test scripts with no false alarms.
Reading PDFs	**Working**	Can pull text out of normal (not scanned) PDF scripts.
Recent fixes	**Done**	Two improvements were just made: it now catches lowercase props (like "the blue ledger") and characters who don't have a normal dialogue label.
The harder, smarter features	**Not done yet**	Several known improvements are intentionally on hold until there's data to test them against.

Bottom line: it's solid and trustworthy on the examples it knows, but its "vocabulary" of patterns is still narrow.

---

4. Where it falls short (explained simply)

In the Scene Connections engine

1. **It only recognizes a few "action" words for props.** It catches "picks up", "has", and "gives" — but not "grabs", "hides", "carries", "drops", etc. So a prop introduced with a different verb can be missed.
2. **Some characters get dropped.** If a name isn't written in a usual way and the language tool doesn't recognize it, the person can quietly disappear from the map.
3. **Capital-letter names confuse the language tool.** Screenplays write names in ALL CAPS, which is exactly the format the underlying tool is worst at reading.
4. **The "what breaks if I cut this" answer can be incomplete** for long chains of setup that pass through several scenes.

In the Story Mistake Finder

1. **It sometimes grabs the wrong words and raises a false alarm.** A real example from a past run: it reported a "job/role contradiction" for the word **"THERE"** with nonsense descriptions. That's the engine misreading a sentence — the kind of thing that makes a user lose trust. **This is the most important thing to fix.**
2. **Its "are two facts similar?" check is unreliable** because it uses a lightweight language model that doesn't really understand word meaning well.
3. **It only knows a few ways to say things.** "Dead" must literally be "dead", "killed", or "died" — it misses "murdered", "passed away", "we lost him".
4. **It can't follow pronouns.** If the script says "he is a doctor", it doesn't know who "he" is, so it misses the fact.
5. **Its sense of time is limited** to days of the week — it misses dates, "the next morning", "that night", and so on.

Across the whole project

- **There's no automatic safety net (automated tests run on every change),** so a future edit could quietly break something.
- **There's no collection of real, "answer-checked" scripts** to measure how accurate the engines actually are.

---

5. What to work on next (in priority order)

Do first — fix the things that hurt trust (quick, low risk)

1. **Stop the false alarms** like the "THERE" example. Teach the mistake finder to ignore filler words and not grab text across sentence boundaries. *(This is the same kind of fix that was just applied successfully to the other engine.)*
2. **Ignore pronouns and filler** when deciding who a fact is about.

Do next — easy wins that catch more (low/medium risk)

1. **Teach it more "action" words for props** (grabs, hides, carries, drops…).
2. **Teach it more ways to say "dead"** (murdered, passed away, killed off…).
3. **Make sure recognized characters always appear on the scene map** (close the "dropped character" gap).

Do after you have test data — the smarter, riskier upgrades

1. **Recognize characters by how they act in a sentence** (the strongest fix for missed names) — but this can misfire (e.g. mistaking a car for a person), so it needs measurement first.
2. **Use a better language model** so the "are these two facts similar?" check actually works.
3. **Follow pronouns** ("he/she/they") so more facts get captured.

Ongoing — foundations

1. **Build a small library of real scripts with the right answers marked** (about 10–20). This is the single most useful step — it unlocks safely doing items 6–8. *(The tool to run and score these already exists; it just needs the marked-up scripts.)*
2. **Add automatic tests** that run on every change, so nothing silently breaks.

---

6. How to proceed — a simple plan

1. **Week 1 — Trust fixes.** Fix the false alarms (items 1–2). These are small, safe, and immediately make the tool feel more reliable.
2. **Week 1–2 — Easy recall wins.** Add the extra action words and "dead" synonyms, and make sure no characters get dropped (items 3–5).
3. **Week 2–3 — Build the test library.** Collect ~10–20 real scripts and mark the real mistakes and important scene links in them (item 9). Use the existing batch tool to score the engines automatically.
4. **Week 3+ — Smart upgrades, measured.** With the test library in place, turn on the riskier improvements one at a time (items 6–8) and check the scores to confirm each one helps rather than hurts.
5. **Throughout — Safety net.** Add the automatic tests (item 10) so every future change is checked.

Guiding rule: fix the false alarms first, then add the safe improvements, then build the test data, and only *then* attempt the clever features — measuring each one so you never trade accuracy for cleverness by accident.

---

7. One-line summary

The engines work well for what they were built to recognize; the next steps are to **stop the occasional**

false alarms, widen what they can recognize, and build a small set of answer-checked scripts so the smarter improvements can be added safely.

End of document.